

Lab 03 – Serial Communication, Data Visualization, and Python–Arduino Integration

Learning Objectives

By the end of this module, students will be able to:

1. Use the Serial Plotter to visualize numeric data sent from Arduino, and explain why text output cannot be plotted.
2. Establish two-way serial communication between a Python script and Arduino using the PySerial library.
3. Control Arduino hardware (an LED) from Python based on external logic — specifically, real-time face detection using OpenCV's Haar Cascade Classifier.
4. Diagnose and resolve common serial communication errors, including port permission conflicts and incorrect use of `Serial.parseInt()` versus `Serial.read()` for continuous data streams.

1. Visualizing Digital Data with Serial Plotter

Theory:

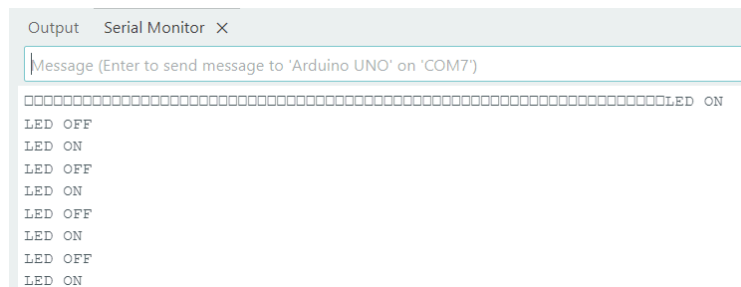
The Serial Plotter (Tools → Serial Plotter in the Arduino IDE) graphs data sent over Serial in real time — but it only understands **numeric values**, one per line. If Arduino sends human-readable text (e.g., "LED ON"), the Plotter cannot interpret it as a data point and will show nothing meaningful. Sending a plain number instead (e.g., 1 or 0) lets the Plotter draw it directly on a graph, with time automatically used as the X-axis.

Arduino also supports a special **label:value** format (no spaces, single colon) that lets the Plotter display a named, color-coded line in a legend — useful once multiple signals are plotted together.

Code — Text Output (Does NOT Plot):

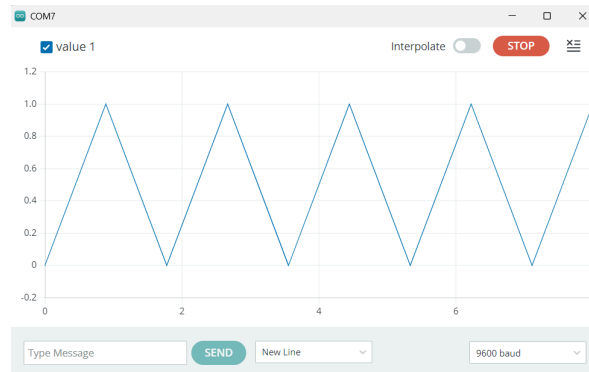
```
void setup() {  
  pinMode(13, OUTPUT);  
  Serial.begin(9600);  
}  
  
void loop() {  
  digitalWrite(13, HIGH);  
  Serial.println("LED ON");  
  delay(1000);  
  
  digitalWrite(13, LOW);  
  Serial.println("LED OFF");  
  delay(1000);  
}
```

Output in Serial Monitor



Code — Numeric Output (Plots Correctly):

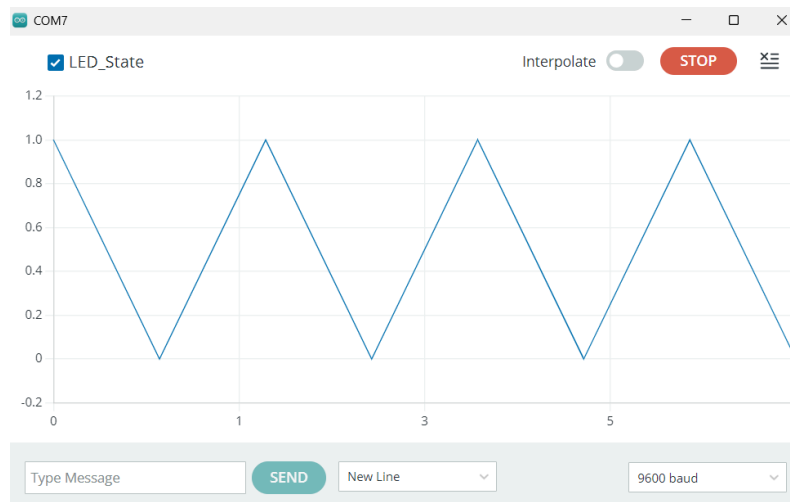
```
void setup() {  
  pinMode(13, OUTPUT);  
  Serial.begin(9600);  
}  
  
void loop() {  
  digitalWrite(13, HIGH);  
  Serial.println(digitalRead(13)); // Sends 1  
  delay(1000);  
  
  digitalWrite(13, LOW);  
  Serial.println(digitalRead(13)); // Sends 0  
  delay(1000);  
}
```



Code — Labeled Numeric Output:

```
void setup() {  
  pinMode(13, OUTPUT);  
  Serial.begin(9600);  
}  
  
void loop() {  
  digitalWrite(13, HIGH);  
  Serial.print("LED_State:");  
  Serial.println(digitalRead(13));  
  delay(1000);  
  
  digitalWrite(13, LOW);  
  Serial.print("LED_State:");  
  Serial.println(digitalRead(13));  
  delay(1000);  
}
```

Output



Explanation:

- `digitalRead(13)` confirms the pin's actual current state rather than assuming it, and returns a clean 1 or 0.
- `Serial.print("LED_State:")` followed immediately by `Serial.println(value)` produces a single line like `LED_State:1` — the exact format the Plotter needs to draw a named trace.
- Mixing text and numbers on the same line (e.g., `"LED ON : 1"`) breaks the parser — the Plotter requires either a bare number or a clean label:value pair with no extra words or spaces.

2. Why a Clean Digital Signal Looks Like a Ramp (Sampling Artifact)

Theory:

A digital pin genuinely switches instantly between HIGH and LOW — on an oscilloscope, this looks like a perfect square wave. However, the Serial Plotter only draws what it receives, and it **connects consecutive points with a straight line**. If only one sample is sent per state (e.g., one point at value 1, then a full second later one point at value 0), the Plotter has no data telling it to "hold flat" in between — so it draws a sloped line between the two points, creating a triangular ramp instead of a square edge.

This is a **sampling/visualization artifact**, not a change in the real electrical signal. Sending multiple samples per second while the pin holds each state gives the Plotter enough flat data points to draw a proper square wave.

Code — Produces a Proper Square Wave (Higher Sampling Rate):

```
void setup() {  
    pinMode(13, OUTPUT);  
    Serial.begin(9600);  
}  
  
void loop() {  
    digitalWrite(13, HIGH);  
    for (int i = 0; i < 20; i++) {           // 20 samples while HIGH  
        Serial.println(digitalRead(13));  
        delay(50);                          // 20 x 50ms = 1000ms total  
    }  
}
```

```
digitalWrite(13, LOW);  
for (int i = 0; i < 20; i++) {      // 20 samples while LOW  
    Serial.println(digitalRead(13));  
    delay(50);  
}  
}
```



Explanation:

- In the first version, only two data points exist per full cycle (one HIGH, one LOW), so the Plotter has to guess what happened in between — resulting in a sloped ramp.
- In the second version, 20 repeated samples at value 1 are sent before switching to 20 samples at value 0, giving the Plotter a flat run of points at each level before the transition — producing a clean stepped/square shape.
- This demonstrates an important general principle: a visualization is only as accurate as its **sampling rate**, even when the underlying signal itself is perfectly clean.

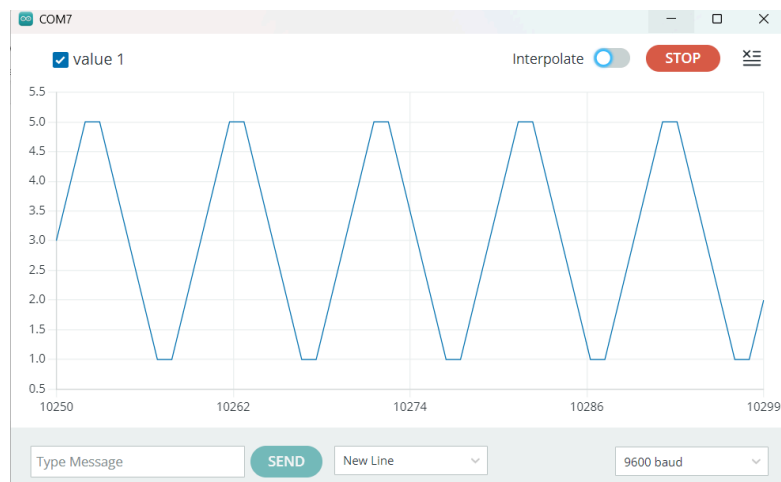
3. Plotting Custom Waveforms — Ramp Up and Down

Theory:

Beyond reading pin states, the Serial Plotter can graph any numeric sequence sent to it — including values generated purely in software. A for loop that counts up, followed by one that counts down, produces a **triangular waveform** when plotted, since each printed value becomes one point on the graph.

Code — Basic Ramp (1 to 5 to 1):

```
void setup() {  
  Serial.begin(9600);  
}  
  
void loop() {  
  // Count up from 1 to 50  
  for (int i = 1; i <= 5; i++) {  
    Serial.println(i);  
    delay(50);  
  }  
  
  // Count down from 50 to 1  
  for (int i = 5; i >= 1; i--) {  
    Serial.println(i);  
    delay(50);  
  }  
}
```



Explanation:

- The first for loop increments i from 1 to 5, printing each value — this draws the rising edge of the triangle.
 - The second for loop decrements i back to 1, drawing the falling edge.
 - delay(50) controls the speed of the ramp; lowering it produces a faster-moving wave, raising it slows the animation down.
 - The Value: label makes this trace identifiable by name once combined with other plotted signals (e.g., alongside an LED state trace).
-

4. Two-Way Communication: Controlling Arduino from Python (PySerial)

Theory:

Serial communication is not limited to Arduino talking to a computer — a Python script can also **send** data to Arduino using the pyserial library, allowing a PC-side program to control hardware in real time. Both sides must agree on the same **baud rate** (e.g., 9600). Since Arduino automatically resets when a new Serial connection is opened, Python must briefly pause (time.sleep(2)) after connecting to give the board time to finish booting before sending data.

Arduino Code (Receiver):

```
void setup() {
  pinMode(13, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  if (Serial.available() > 0) {
    int value = Serial.parseInt();    // Reads an incoming number
    if (value == 1) {
      digitalWrite(13, HIGH);
    } else if (value == 0) {
      digitalWrite(13, LOW);
    }
  }
}
```

Python Code (Sender):

```
import serial
import time
```

```
arduino = serial.Serial(port='COM7', baudrate=9600, timeout=1)
time.sleep(2) # Wait for Arduino to reset after connecting

while True:
    arduino.write(b'1')      # Turn LED ON
    print("Sent: 1")
    time.sleep(2)

    arduino.write(b'0')      # Turn LED OFF
    print("Sent: 0")
    time.sleep(2)
```

Explanation:

- `Serial.available()` checks whether incoming bytes have arrived; `Serial.parseInt()` reads and converts them into a number.
- `arduino.write(b'1')` sends the raw byte for the character '1' from Python to Arduino over the same USB connection.
- This pattern — Python decides *what* to do, Arduino executes the *physical* action — is the foundation for combining Python-side logic (sensors, algorithms, computer vision) with real hardware control.

5. Automating Hardware with Computer Vision: Face-Detection-Triggered LED

Theory:

A **Haar Cascade Classifier** is a pretrained object-detection model that scans an image at multiple scales and positions, using a sequence of simple feature checks ("cascades") to quickly reject non-matching regions and zero in on likely matches — in this case, human faces. OpenCV ships pretrained cascade files internally (accessed via `cv2.data.harcascades`), so no separate XML download is required. Combining this with PySerial allows a webcam feed to directly drive a physical LED: face detected → LED ON, no face → LED OFF.

Arduino Code (unchanged from Section 4 receiver — but see Section 6 for an important fix):

```
void setup() {
```

```

pinMode(13, OUTPUT);
Serial.begin(9600);
}

void loop() {
    if (Serial.available() > 0) {
        char value = Serial.read();    // Reads one raw character
        if (value == '1') {
            digitalWrite(13, HIGH);
        } else if (value == '0') {
            digitalWrite(13, LOW);
        }
    }
}
}

```

Python Code (Face Detection + Serial Control):

```

import cv2
import serial
import time

# --- Setup Serial connection to Arduino ---
arduino = serial.Serial(port='COM7', baudrate=9600, timeout=1)
time.sleep(2) # wait for Arduino to reset after connecting

# --- Load OpenCV's built-in Haar Cascade for face detection ---
face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + 'haarcascade_frontalface_default.xml'
)

# --- Start webcam ---
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(
        gray, scaleFactor=1.1, minNeighbors=5, minSize=(60, 60)
    )

    if len(faces) > 0:
        for (x, y, w, h) in faces:
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
            arduino.write(b'1')
            #print("Face detected!")
    else:

```

```
    arduino.write(b'0')
    #print("No face detected.")

cv2.imshow("Face Detection", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
arduino.close()
```

Explanation:

- cv2.data.harcascades points to OpenCV's own bundled cascade files — no manual XML download needed.
- detectMultiScale() scans the grayscale frame for face-like patterns; minNeighbors reduces false positives, and minSize filters out unlikely tiny detections.
- The webcam loop runs continuously at close to the camera's native frame rate, sending '1' or '0' every frame based on whether any face was found.
- Note that Arduino uses Serial.read() here instead of Serial.parseInt() — see Section 6 for why this specific change is essential for a continuous, unbroken stream of characters.

6. Common Serial Communication Errors and Debugging

Theory — Port Access Conflicts:

A Serial (UART) port represents a **single physical communication channel** — one TX wire, one RX wire. Only one program can hold it open at a time; if two programs try to open the same COM port simultaneously, the operating system rejects the second request with a PermissionError / Access is denied. This is why the Arduino IDE's Serial Monitor and a Python script cannot both be connected at once — whichever opens the port first "owns" it until it's closed.

Theory — Serial.parseInt() vs. Serial.read():

`Serial.parseInt()` is designed for input with natural pauses or terminators (like a human typing into Serial Monitor and pressing Enter) — it keeps accumulating digits as long as more arrive within its timeout window. When Python sends a rapid, continuous, unseparated stream of '1's and '0's (e.g., 30 times per second from a webcam loop), `parseInt()` can misread this as one long accumulated number instead of separate values, so `(value == 1)` never correctly matches. Switching to `Serial.read()` — which reads exactly one raw character per call with no accumulation or waiting — correctly handles a continuous character stream.

Troubleshooting Reference Table

Symptom	Likely Cause	Fix
PermissionError: Access is denied (Python)	Arduino IDE Serial Monitor/Plotter still open	Close Arduino IDE completely before running Python
avrdude: ser_open(): Access is denied (Arduino IDE)	Python script still running / didn't close port	Stop Python script, end lingering python.exe processes
LED never turns on despite correct wiring	<code>Serial.parseInt()</code> misreading a continuous character stream	Switch Arduino to <code>Serial.read()</code> for single-character reads

Class Tasks

Task 1: Modify the numeric LED-state code (Section 1) to also plot the state of a second digital pin (e.g., a button), using two different label:value names so both traces appear on the same graph.

Task 2: Using the low-sampling-rate code (Section 2), predict on paper what the Plotter will show if `delay(1000)` is changed to `delay(2000)`. Then run it and confirm your prediction.

Task 3: Modify the ramp code (Section 3) so the waveform rises and falls **twice as fast**, and explain exactly which line(s) you changed.

Task 4: Extend the Python-to-Arduino control code (Section 4) so that sending the value 2 sets the LED to a **dim brightness** using `analogWrite()`, in addition to the existing ON/OFF behavior.

Task 5: Run the face-detection code (Section 5) and record how many frames per second your webcam loop achieves. Then re-add the "only send on state change" check from earlier discussions and compare the Serial traffic and responsiveness.

Task 6 (Challenge): Deliberately revert Arduino's code to use `Serial.parseInt()` instead of `Serial.read()` while running the face-detection script, and observe the LED's faulty behavior. Explain in your own words, using what you now know about `parseInt()`, why the LED fails to respond correctly.

Task 7 (Challenge): Deliberately trigger a `PermissionError` by opening Arduino IDE's Serial Monitor while your Python script is running. Take a screenshot of the error, then correctly resolve it and explain each step you took.

Conclusion

Through this module, you have learned how to:

- Send and correctly format numeric data for visualization on the Serial Plotter, including named label:value traces.
- Recognize and fix a sampling-related visual artifact that makes a clean digital signal appear as a ramp instead of a square wave.
- Generate custom software-defined waveforms and plot them in real time.
- Establish two-way communication between Python and Arduino using `PySerial`, enabling a PC-side program to control physical hardware.
- Integrate computer vision (OpenCV Haar Cascade face detection) with embedded hardware control to build a real-time automation pipeline.
- Diagnose and resolve real-world serial communication issues, including port conflicts and the critical difference between `Serial.parseInt()` and `Serial.read()` for continuous data streams.

This module bridges embedded systems and higher-level software (Python, computer vision) — a foundational skill for building intelligent, sensor-driven robotics and IoT systems that respond to the real world in real time.